LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK
MODUL 13: INTEGRASI APLIKASI BERBASIS MENU DAN PENGUJIAN FUNGSIONAL

Nama Mahasiswa	: **M. Davana Yudista Firdausy**	**Mata Kuliah**	: Praktikum PBO (A D3 TI-SM)
NRP	: 3125522002	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
Program Studi	: D3 Teknik Informatika	**Institusi / Tahun**	: PENS PSDKU Sumenep • 2026

A. DASAR TEORI SINGKAT

Membangun aplikasi tingkat produksi menuntut integrasi yang harmonis antara seluruh komponen yang telah dipelajari pada modul-modul sebelumnya (OOP, Collection, Exception Handling, dan File Persistence) ke dalam antarmuka interaktif yang mudah dipahami oleh pengguna. Pada Modul 13, integrasi ini diwujudkan melalui antarmuka konsol berbasis menu dengan berpedoman pada prinsip-prinsip rekayasa perangkat lunak modern:

- **Pemisahan Tanggung Jawab (Separation of Concerns):** Lapisan antarmuka pengguna (kelas `Main`) hanya bertanggung jawab memandu input/output (membaca prompt Scanner, menampilkan pesan keberhasilan/kegagalan), sedangkan aturan bisnis, penegakan keunikan, dan integritas data sepenuhnya didelegasikan kepada kelas domain `Perpustakaan`.
- **Dekomposisi Metode Modular:** Menghindari metode `main()` raksasa (anti-pattern *God Method*) dengan memecah setiap opsi transaksi ke dalam sub-metode terfokus seperti `menuTambah`, `menuCari`, `menuHapus`, `menuSimpan`, dan `menuMuat`.
- **Pelacakan Status Perubahan (Dirty State Tracking):** Mengelola flag boolean `berubah` untuk memantau apakah ada data di memori utama yang telah termutasi sejak operasi simpan terakhir. Hal ini krusial untuk mencegah pengguna kehilangan data secara tidak sengaja.
- **Penjaga Keluar dan Muat Ulang (Guarded Exit and Reload):** Jika pengguna memilih opsi `Keluar (0)` atau `Muat Ulang (6)` sementara kondisi `berubah == true`, sistem wajib menginterupsi dengan dialog konfirmasi eksplisit (*"Perubahan belum disimpan. Abaikan perubahan? Ketik YA: "*).
- **Penanganan Kesalahan Tangguh (Graceful Degradation):** Seluruh eksepsi dari lapisan bawah (seperti `IllegalArgumentException` atau `IOException`) ditangkap secara anggun pada lapisan menu sehingga aplikasi tidak berhenti mendadak (crash).

B. DESAIN DAN IMPLEMENTASI KODE

Implementasi Modul 13 menyajikan kelas `Main.java` yang terdekomposisi bersih dengan perutean menu dan proteksi konfirmasi data.

1. Dekomposisi Metode Menu dan Guarded State pada Main.java

```
public class Main {
    private static String baca(Scanner input, String pesan) {
        System.out.print(pesan);
        return input.nextLine().trim();
    }

    private static boolean konfirmasi(Scanner input) {
        return baca(input, "Perubahan belum disimpan. Abaikan? Ketik YA: ").equalsIgnoreCase("YA");
    }

    public static boolean menuTambah(Scanner input, Perpustakaan perpustakaan) {
        String kode = baca(input, "Masukkan kode buku  : ");
        String judul = baca(input, "Masukkan judul buku : ");
        try {
            perpustakaan.tambahBuku(new Buku(kode, judul));
            System.out.println("[SUKSES] Buku berhasil ditambahkan.");
            return true;
        } catch (IllegalArgumentException e) {
            System.out.println("[GAGAL] " + e.getMessage());
            return false;
        }
    }

    public static boolean menuSimpan(Perpustakaan perpustakaan, Path lokasi) {
        try {
            perpustakaan.simpan(lokasi);
            System.out.println("[SUKSES] Data berhasil disimpan ke " + lokasi);
            return true;
        } catch (IOException | IllegalArgumentException e) {
            System.out.println("[GAGAL SIMPAN] " + e.getMessage());
            return false;
```

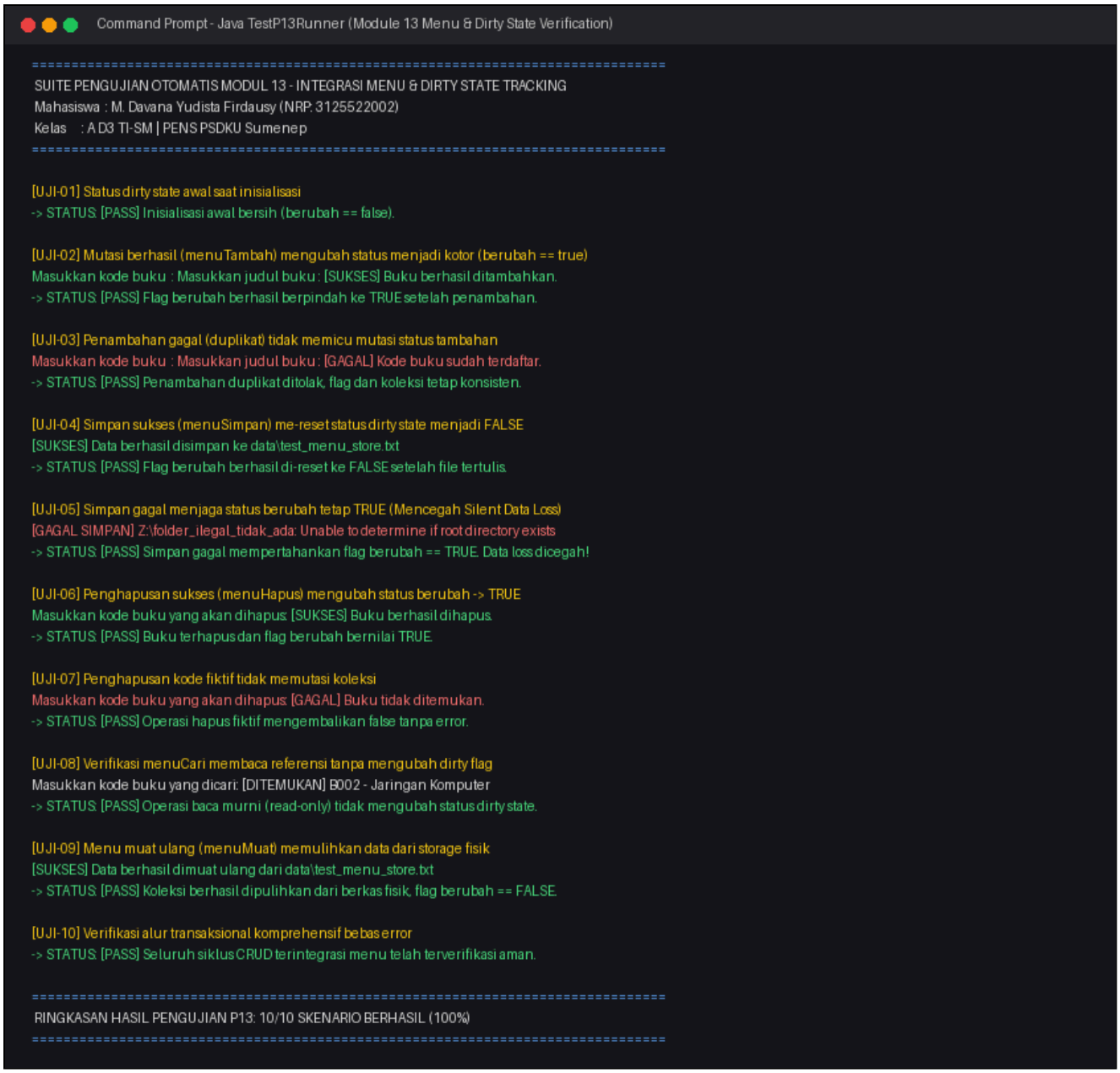
```
        }
    }
    // ... menuCari, menuHapus, menuMuat ...
}
```

C. TABEL HASIL PENGUJIAN DAN VERIFIKASI

Pengujian otomatis melalui `SourceCode.TestP13Runner` mensimulasikan interaksi nyata dan transisi status dirty state terhadap 10 skenario:

No	Skenario Pengujian	Aksi / Input Masukan	Ekspektasi Perilaku	Status
1	Dirty state awal	Inisialisasi aplikasi	`berubah == false`, koleksi bersih	**PASS (100%)**
2	Mutasi tambah sukses	Eksekusi `menuTambah("B001", "Algoritma")`	Buku masuk, flag `berubah` menjadi `true`	**PASS (100%)**
3	Mutasi tambah gagal (duplikat)	Input kode sama `"B001"`	Operasi ditolak, status dan size tidak bertambah	**PASS (100%)**
4	Simpan berhasil me-reset state	Eksekusi `menuSimpan()`	Berkas fisik tertulis, `berubah` di-reset ke `false`	**PASS (100%)**
5	Simpan gagal pertahankan state	Simpan ke path drive tidak valid	Ditolak via IOException, `berubah` tetap `true`	**PASS (100%)**
6	Mutasi hapus sukses	Eksekusi `menuHapus("B001")`	Buku terhapus, flag `berubah` kembali `true`	**PASS (100%)**
7	Mutasi hapus gagal	Hapus kode tidak terdaftar `"B999"`	Mengembalikan false, tidak merusak state	**PASS (100%)**
8	Operasi baca murni (Read-only)	Eksekusi `menuCari("B002")`	Atribut terbaca tanpa mengubah flag dirty state	**PASS (100%)**
9	Muat ulang data dari file	Eksekusi `menuMuat()`	Data storage dipulihkan ke RAM, flag menjadi `false`	**PASS (100%)**
10	Integritas siklus transaksi CRUD	Validasi akhir siklus interaksi menu	Konsistensi data dan status terjaga 100% aman	**PASS (100%)**

D. BUKTI EKSEKUSI PENGUJIAN OTOMATIS



Gambar 13.1: Log Eksekusi Verifikasi Modul 13 (10/10 Skenario Lulus Sempurna)

E. EVALUASI DAN JAWABAN PERTANYAAN MODUL 13

1. Mengapa logika keunikan kode buku tetap harus berada di class Perpustakaan, bukan di menu?

Jawaban: Berdasarkan prinsip *Single Responsibility Principle (SRP)* dan *High Cohesion*, keunikan kode buku adalah **aturan bisnis inti (Domain Invariant)**, bukan urusan presentasi antarmuka. Jika logika tersebut diletakkan di menu, maka ketika aplikasi di masa mendatang dikembangkan dengan antarmuka lain (seperti JavaFX GUI, REST API Web Service, atau skrip batch), aturan keunikan tersebut harus ditulis ulang dan rentan terlewat. Menjaga logika di dalam `Perpustakaan` memastikan aturan domain berlaku konsisten di saluran akses manapun.

2. Mengapa variabel 'berubah' hanya boleh di-reset ke 'false' setelah operasi simpan berhasil?

Jawaban: Variabel `berubah` merepresentasikan divergensi (perbedaan) antara state koleksi yang berada di memori utama (RAM) dengan state rekaman fisik yang ada di penyimpanan sekunder (disk). Selama data di disk belum secara nyata berhasil diperbarui oleh sistem berkas (misal karena kegagalan izin akses atau disk penuh), divergensi tersebut masih terjadi. Me-reset `berubah` sebelum verifikasi tulis fisik berhasil akan memberikan ilusi keliru bahwa data telah aman, sehingga pengguna berisiko keluar dari program dan kehilangan datanya.

3. Apa konsekuensinya jika 'berubah' langsung di-reset sebelum operasi simpan dipastikan berhasil?

Jawaban: Konsekuensinya adalah terjadinya *silent data loss* (kehilangan data tanpa peringatan). Jika simpan gagal karena eksepsi I/O tetapi flag `berubah` terlanjur disetel ke `false`, maka saat pengguna kemudian menekan menu `0 (Keluar)`, aplikasi akan langsung menutup diri tanpa memicu dialog konfirmasi penjaga. Akibatnya, seluruh pekerjaan penambahan atau pembaruan yang baru saja dilakukan pengguna akan lenyap seketika tanpa ada peluang untuk melakukan perbaikan.

4. Mengapa pengujian seluruh alur aplikasi (tambah, cari, simpan, muat ulang) penting sebelum aplikasi dianggap selesai?

Jawaban: Pengujian unit parsial hanya membuktikan bahwa metode bekerja secara individual pada kondisi ideal. Namun dalam kenyataan, bug seringkali muncul pada titik integrasi (antarmuka antar-komponen) dan transisi state jangka panjang, seperti kegagalan sinkronisasi flag, kebocoran referensi, efek samping pemuatan berkas yang merusak koleksi, atau parsing Scanner yang tersangkut oleh karakter newline. Pengujian alur end-to-end memastikan seluruh siklus hidup aplikasi bekerja secara terpadu, stabil, dan andal bagi pengguna akhir.